

Úvod do databázových systémů

Kapitola 5
Logický model

Logický model

- druhá vrstva databázového modelování
 - nižší úroveň abstrakce než konceptuální model
 - vyšší úroveň abstrakce než fyzický model
 - řeší logické struktury, v nichž jsou data uložena, vlastní implementaci těchto struktur ponechává na fyzickém modelu
 - řeší manipulaci s daty ve strukturách (operace)

Logický model

- různé log. modely
 - síťový (60. léta)
 - datové prvky vzájemně propojeny ukazateli
 - hierarchický
 - předpokládá stromovou strukturu datových prvků
 - relační (70. léta)
 - objektový (80. léta)
 - data modelována třídami, instancemi jsou objekty
 - de facto rozšíření síťového modelu
 - objektově-relační (90. léta)
 - hybrid, komerčně úspěšný, podpora ve standardech
 - XML
 - obsahuje prvky hierarchického modelu

Logický model

- volba databázového modelu určuje prostředky pro vytváření struktury databáze (DDL) a prostředky pro tvorbu aplikací (DML, dotaz. jazyk, TCL, DCL)
 - pro určitý typ aplikace můžeme výběrem vhodného DB modelu mnoho ušetřit
 - volbu DB modelu je třeba dobře uvážit
- příklady
 - relační model - SQL
 - objektový model - OQL
 - XML model - Xpath, XQuery

Relační model dat

- používá jediný konstrukt – relaci
- RMD se objevil v roce 1970 v článku E.F. Codd
 - odděluje data od jejich implementace
 - data jsou uložena v abstraktní matematické struktuře
 - při manipulaci s daty se nezajímáme o konkrétní mechanismy přístupu k datovým prvkům
 - dva prostředky manipulace s daty
 - relační algebra
 - relační kalkul
 - normalizace relací
 - posouzení kvality návrhu databáze

Relace

- základní principy
 - charakteristika dané relace se nazývá **schema**
 - jméno relace, jména atributů, domény atributů
 - prvky domén jsou atomické hodnoty (1.normální forma)
 - specifická integritní omezení (která neexistují v ER modelu)
 - primární klíč
 - cizí klíč

Relace

- matematicky

- mějme systém neprázdných množin (domén)

$$D_i, 1 \leq i \leq n$$

- podmnožina R kartézského součinu

$$R \subseteq D_1 \times D_2 \times D_3 \times \dots \times D_n$$

se nazývá **n-ární relace** (n – řád relace)

- relace je tedy také množina

- prvky této relace jsou **uspořádané n-tice**

$$(d_1, d_2, d_3, \dots, d_n) \text{ kde } d_i \in D_i$$

- v DB nemá smysl uvažovat nekonečné relace, proto se omezíme na konečné podmnožiny kart. součinu

Relace

■ příklad

■ mějme množiny $A = \{1, 2, 3\}$ a $B = \{x, y\}$

■ kartézský součin těchto množin je

$$A \times B = \{ (1,x), (1,y), (2,x), (2,y), (3,x), (3,y) \}$$

■ použijeme-li vícenásobný kartézský součin:

$$A \times B \times B =$$

$$\{ (1,x,x), (1,y,x), (2,x,x), (2,y,x), (3,x,x), (3,y,x), \\ (1,x,y), (1,y,y), (2,x,y), (2,y,y), (3,x,y), (3,y,y) \}$$

■ $A \times B \times B \times B =$

$$\{ (1,x,x,x), (1,y,x,x), (2,x,x,x), (2,y,x,x), (3,x,x,x), (3,y,x,x), \\ (1,x,y,x), (1,y,y,x), (2,x,y,x), (2,y,y,x), (3,x,y,x), (3,y,y,x), \\ (1,x,x,y), (1,y,x,y), (2,x,x,y), (2,y,x,y), (3,x,x,y), (3,y,x,y), \\ (1,x,y,y), (1,y,y,y), (2,x,y,y), (2,y,y,y), (3,x,y,y), (3,y,y,y) \}$$

Relace

- příklad

- a tak dále

- každý další součin s další doménou nám přidá další prvek do n-tic, vždy budou ve výsledku všechny možné kombinace prvků (hodnot z domén)

- součin $A \times B \times B \times B$ má tedy prvky – čtveřice, kde na první místě je nějaký prvek z A , na dalších místech pak libovolné prvky z B

Relace

■ příklad

- celý součin $A \times B \times B \times B$

lze zobrazit jako tabulku

- sloupce pojmenujeme třeba S_1 až S_4
- domény sloupců jsou A,B,B,B
- v řádcích jsou jednotlivé čtveřice obsahující všechny možné kombinace hodnot z domén

$S_1:A$	$S_2:B$	$S_3:B$	$S_4:B$
1	x	x	x
1	y	x	x
2	x	x	x
2	y	x	x
3	x	x	x
3	y	x	x
1	x	y	x
1	y	y	x
2	x	y	x
2	y	y	x
3	x	y	x
3	y	y	x
1	x	x	y
1	y	x	y
2	x	x	y
2	y	x	y
3	x	x	y
3	y	x	y
1	x	y	y
1	y	y	y
2	x	y	y
2	y	y	y
3	x	y	y
3	y	y	y

Relace

■ příklad

- použijme místo školometských domén A a B nějaké praktičtější – třeba datové typy INT a CHAR(10)
 - necht' INT může mít hodnoty od -32768 do 32767
 - CHAR(10) je desetiznakový řetězec libovolných znaků
- součin $\text{INT} \times \text{CHAR}(10) \times \text{CHAR}(10) \times \text{CHAR}(10)$ bude tedy množina čtveřic:
 - první položka je celé číslo z intervalu $\langle -32768, 32767 \rangle$
 - druhá až čtvrtá položka jsou řetězce deseti znaků
 - všech možných kombinací bude velmi mnoho
 - jestliže znak je reprezentován jedním bytem a INT dvěma byty, celá čtveřice bude obsahovat $2 + 3 \cdot 10 = 32$ bytů, tj. celý kartézský součin bude obsahovat 256^{32} prvků

Relace

■ příklad

- součin $\text{INT} \times \text{CHAR}(10) \times \text{CHAR}(10) \times \text{CHAR}(10)$
tedy může obsahovat mimo jiné takovéto čtveřice

$S_1:\text{INT}$	$S_2:\text{CHAR}(10)$	$S_3:\text{CHAR}(10)$	$S_4:\text{CHAR}(10)$
1	x#\$%abscde	desetznaků	řetězecXYZ
2	1234567890	desetznaků	řetězecABC
118	0987654321	desetznaků	řetězecPRS
30000	abcdefghij	desetznaků	řetězecKOS
-97	-----%%%%%%%%	desetznaků	řetězecSUK
3	GKJPIUTR46	desetznaků	řetězecSŮL
777	ýáůú/][2yu	desetznaků	řetězecREK
-256	\é087ž^&	desetznaků	řetězecBLB

Relace

■ příklad

- součin $\text{INT} \times \text{CHAR}(10) \times \text{CHAR}(10) \times \text{CHAR}(10)$
ale může obsahovat i smysluplnější čtveřice
(pozn.: krátké řetězce doplníme nevýznamnými mezerami)

číslo_zam: INT	jméno: CHAR(10)	příjmení: CHAR(10)	titul: CHAR(10)
1	Jan	Paukert	Mgr.
2	Eleonora	Frauenberg	Dr.
3	Miloš	Lošák	
4	Pavel	Velek	
5	Jarmila	Milá	prof. Ing.
6	Stanislava	Slavatová	Bc.
7	Cy	Cohen	
8	Li	C'	

Relace

- a tak jsme přešli od obecné relace k tabulce zaměstnanců...
- **otázka** – jak zařídit, aby řádky této tabulky (tj. prvky relace) neobsahovaly kombinaci
(1, x#\$%abscde, desetznaků, řetězecXYZ)
ale pouze smysluplné kombinace, jako třeba
(1, Jan, Paukert, Mgr.)
- **odpověď** – pomocí integritních omezení!
 - n-tice, které budou vyhovovat podmínkám, se nazývají přípustné

Relace

- souvislost relace a tabulky - shrnutí
 - relaci si představíme jako tabulku s konečným počtem řádků a s n sloupci
 - každému prvku relace odpovídá jeden řádek
 - buňky ve sloupci k mohou nabývat pouze hodnot z domény D_k
 - sloupcům přiřadíme (různá) jména A_i
 - schema relace R (záhlaví tabulky) – lineární zápis
 $R(A_1:D_1, A_2:D_2, \dots, A_n:D_n)$ anebo bez domén
 $R(A_1, A_2, \dots, A_n)$ anebo
 $R(\mathbf{A})$, kde $\mathbf{A} = \{A_1, A_2, \dots, A_n\}$
např. OSOBA(RČ, jméno, příjmení, bydliště)

Relace

- rozdíly mezi tabulkou a relací
 - tabulka může obsahovat stejné řádky, relace nikoli (je to množina, jeden prvek je v ní jednou)
 - v relaci nejsou prvky uspořádané, tabulka má vždy nějak uspořádané řádky
 - je nutná jistá obezřetnost při zacházení s pojmy relace a tabulka, neboť nejsou ekvivalentní

Relace

- nástin souvislosti relace s entitou v E-R modelu (detailně bude popsáno v dalším)
 - schema relace $\sim$ entitní typ
 - prvek relace (n-tice) $\sim$ výskyt entity
 - prvek n-tice $\sim$ atribut dané entity
- souvislost tabulky s entitou
 - záhlaví tabulky $\sim$ entitní typ
 - řádek tabulky $\sim$ výskyt entity
 - sloupec tabulky $\sim$ atribut
- relace odpovídá entitnímu typu lépe než tabulka
 - unikátnost prvků (evidence osob: dvojníci?)
 - neuspořádanost prvků

Relace

■ základní operace

- vytvoř novou relaci (tabulku)
- přidej novou n-tici (řádek) do dané relace (tabulky)
- vymaž n-tice (řádky) zadaných vlastností
- ve vybraných n-ticích (řádcích) zadané relace (tabulky) změň hodnoty zadaných prvků (polí)
- vytvoř novou relaci (tabulku) ze zadané relace:
 - výběrem n-tic (řádků) zadaných vlastností: **selekce**
 - výběrem zadaných atributů (sloupců): **projekce**
- vytvoř novou relaci (tabulku) ze zadaných relací (tabulek) pomocí
 - množin. operací **sjednocení, průnik, rozdíl, kart. součin**
 - operace **spojení**

1. normální forma

- normální forma (NF) – nějaké omezení struktury a závislostí mezi jednotlivými atributy
- 1. NF
 - každý datový prvek (hodnota buňky v tabulce) je atomický, tj. dále nedělitelný
 - srovnej speciální typy atributů (vícehodnotový a skupinový) v konceptuálním modelu
 - základní předpoklad RMD, bohužel někdy je omezující
 - řešením jsou např. objektové nebo objektově-relační databáze

Integritní omezení

- schematem relace popisujeme jen samotnou datovou strukturu
- musíme tedy zajistit, aby se do relací dostala pouze správná data => **integritní omezení**
 - stejně jako v konceptuálním modelu, i zde jsou IO obvykle logické podmínky na data
 - některá IO logický model dědí z konceptuálního modelu, některá IO vznikají až na úrovni logického modelu

Integritní omezení

- relace, jejíž n-tice vyhovují všem IO se nazývá **přípustná**
- **základní IO**
 - unikátnost n-tic (u množinového pojetí není třeba zdůrazňovat – je implicitní, ale u tabulek ne)
 - rozsah hodnot atributu je dán typem (doménou)
 - existence **klíče** schématu
- **další IO**
 - referenční integrita (cizí klíče)
 - obecné podmínky na data (věk > 0 apod...)

Integritní omezení

- klíč schematu relace $R(\mathbf{A})$
 - minimální množina atributů z $\mathbf{A}$, jejichž hodnoty jednoznačně určují prvky relace R
 - minimalita – nelze odebrat žádný atribut, aniž by to narušilo identifikační schopnost
 - máme-li dvě různé n -tice z přípustné relace R , pak existuje alespoň jeden klíčový atribut, jehož hodnota se pro tyto dvě n -tice liší
- je-li klíčem jediný atribut, nazývá se jednoduchý, ostatní jsou složené
- atribut, který je součástí nějakého klíče, je **klíčový**, jinak je neklíčový

Integritní omezení

- klíč schematu relace $R(\mathbf{A})$
 - kandidátů na klíč může být v relaci víc, vybíráme jeden, který označujeme jako **primární**
 - v nejhorším případě jsou klíčem všechny atributy relace R
 - připojíme-li ke klíči libovolný další atribut, naruší se minimalita, ale nikoli identifikační schopnost, takové množině atributů říkáme **nadklíč**
 - to, že množina nemůže obsahovat duplicitní prvky, je přímo důsledkem definice klíče
 - pozn.: SQL podporuje tzv. kolekci n-tic, kde jsou povoleny duplicitní řádky, to ale není relace, na relaci to převedeme klauzulí DISTINCT, viz další přednášky

Integritní omezení

- převod IO konceptuálního schematu (např. kardinalita, parcialita) do logického modelu bude popsán v dalších přednáškách
- jedním z integritních omezení v RMD, které tento převod podporuje je **referenční integrita**
 - popisuje vztah mezi daty ve dvou relacích
 - např. most musí ležet na nějaké silnici
 - relace MOST má atribut (skupinu atributů) – **cizí klíč**, jehož hodnota musí být přítomna v relaci SILNICE jako primární klíč

Integritní omezení

referenční integrita – příklad

Zboží (Název: string, Výrobce:string, Cena: float, Skladem: int),
IO(Název je klíč, Výrobce je cizí klíč do tabulky Výrobce(Název))

Název	Výrobce	Cena	Skladem
Mouka	Odkolek	5,80	100
Chleba	Odkolek	12,50	56
Ariel	P&G	325,-	15
Košťě	Kartáčovny	135,-	32
Hřebík	Ferona	0,70	9000
Šroub	Ferona	0,90	3600

klíč

cizí klíč

(klíč v tabulce Výrobce)

Výrobce (Název: string, Adresa:string, Dlužíme: bool)
IO(Název je klíč)

Název	Adresa	Dlužíme
Odkolek	Praha	ANO
Kartáčovny	Liberec	NE
Ferona	Olomouc	NE
P&G	USA	ANO
VelkoMasna	Beroun	NE
Papírna	Šumperk	NE

klíč

Integritní omezení

- příklad

- KINO(název_k, adresa)
- FILM(jméno_f, herec, rok)
- PROGRAM(název_k, jméno_f, datum)

- integritní omezení

- primární klíče (podtrženě)
- cizí klíče
 - $\text{PROGRAM}[\text{jméno_f}] \subseteq \text{FILM}[\text{jméno_f}]$
 - $\text{PROGRAM}[\text{název_k}] \subseteq \text{KINO}[\text{název_k}]$
- daný film se v daném kině dává jen jednou
- v kině se nehraje více než dvakrát týdně
- jeden film se nehraje více než ve třech kinech

Ošetření integritních omezení

- mnohá IO lze definovat na úrovni DB, není třeba se spoléhat na aplikační programy
 - deklarativní při vytvoření schématu databáze – ideální, avšak pro koncového uživatele nestačí, nepohodlné
 - hlídá je automaticky DBMS
 - výše jsme zavedli primární klíče a cizí klíče
 - později v SQL přibudou UNIQUE, NOT NULL a CHECK
- procedurální na straně klienta
 - kontrola probíhá pouze na úrovni aplikace, potenciální zdroj chyb
 - doplněk k prvnímu způsobu
- procedurální na straně serveru
 - moduly, uložené v databázi, které provádí server
 - trigger – procedury vázané na jisté události v DB

Objektový model dat

- objekt = data + metody
- mezi objekty existuje více typů vztahů
 - skládání
 - dědění
 - závislost
 - klasifikace podle tříd
- strukturované informace není třeba rozdělovat jako v relačním modelu

Objektový model dat

- protokol objektu je dán množinou přístupných zpráv (ne atributů jako v RMD)
- jedna množina objektů může s využitím polymorfismu obsahovat objekty s různou strukturou dat i metod
- identita objektu je dána nejen vnitřními, ale i vnějšími vazbami, klíče jsou interní záležitostí

Objektový model dat

- konstrukty

- **objekt**

- generován jako instance dané třídy (která nese informace o jménech atributů, specifikaci domén atributů, názvech metod, ...)
 - má stav (hodnoty atributů)

- množinové konstrukce – **kolekce**

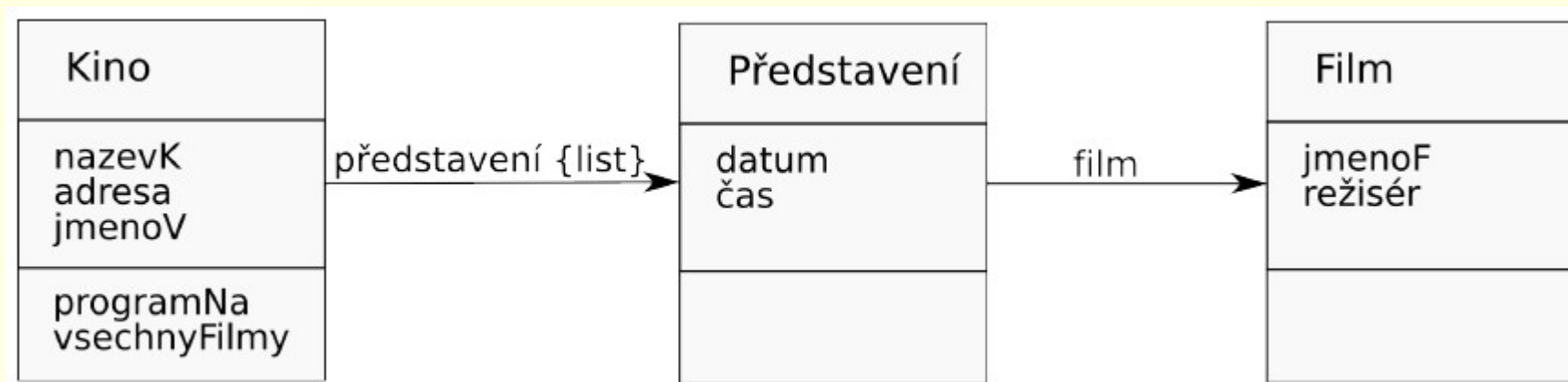
- set, bag, list, array, dictionary, ...

- množinové **operace**

- so:, select:, collect:, detect:, inject:, reject:, intersect:, union:, ...

Objektový model dat

■ příklad



■ metody objektu Kino:

programNa: datum

$\wedge$ predstaveni select: [:p | p datum = datum]

vsechnyFilmy

$\wedge$ (predstaveni collect: [:p | p film]) as Set

XML model dat

- podobá se hierarchickému modelu
 - XML dokument je obvykle chápán jako strom
- datový model : elementy, atributy, ...
- silné a standardizované dotazovací jazyky (XPath,XQuery)
- mnoho implementací a mnoho věcí stále ve vývoji (indexování, zamykání, ...)

XML model dat

■ příklad schematu

```
<!ELEMENT program (kino*)>
<!ELEMENT kino (nazev_k,  adresa, hraje*)>
<!ELEMENT hraje (film, datum)>
<!ELEMENT film (nazev_f, herec, rok)>
<!ELEMENT nazev_k (#PCDATA)>
<!ELEMENT adresa (#PCDATA)>
<!ELEMENT datum (#PCDATA)>
<!ELEMENT nazev_f (#PCDATA)>
<!ELEMENT herec (#PCDATA)>
<!ELEMENT rok (#PCDATA)>
```

■ mnoho opakujících se hodnot

- nevhodné pro manipulaci s daty

■ vhodné pro generování reportů, tisk programu apod.

XML model dat

■ příklad dat

```
<program>
  <kino>
    <nazev_k> MAT </nazev_k>
    <adresa> Karlovo nám. 18, Praha 2 </adresa>
    <hraje>
      <film>
        <nazev_f> Forest Gump </nazev_f>
        <herec> Tom Hanks </herec> <rok> 1998 </rok>
      </film>
      <datum> 3.1. 2007 </datum>
      <film>
        <nazev_f> Vratné láhve </nazev_f>
        <herec> Zdeněk Svěrák </herec> <rok> 2006 </rok>
      </film>
      <datum> 17. 5. 2007 </datum>
    </hraje>
  </kino>
  <kino> ... </kino>
  ...
</program>
```